



Linguagem de Programação IV

Marcio C. B. Pache
marcio.pache@ifms.edu.br

Características Avançadas de Programação para Web -Parte 1-



Arquiteturas e Padrões Avançados

1. SPA (Single Page Applications)
2. MPA (Multi Page Applications)
3. SSR (Server-Side Rendering)
4. SSG (Static Site Generation)
5. PWA (Progressive Web Apps)
6. Componentização
7. Micro Frontends



Arquiteturas e Padrões Avançados

- Essas abordagens surgiram para resolver problemas de performance, escalabilidade, experiência do usuário e manutenção em aplicações web modernas.
- Cada padrão tem um propósito específico e pode ser usado isoladamente ou combinado com outros.



Arquiteturas e Padrões Avançados

- **SPA (Single Page Applications)**
- Uma SPA é uma aplicação web que carrega uma única página HTML e atualiza o conteúdo dinamicamente via JavaScript, sem recarregar a página inteira a cada navegação.
- Melhor experiência do usuário (navegação rápida), mas pode exigir mais atenção com SEO.
 - **SEO-Otimização para Motores de Busca**, é o conjunto de práticas para melhorar o posicionamento de um site nos resultados de pesquisa orgânica (não paga) em mecanismos como o Google.
 - O objetivo é aumentar a visibilidade do site e atrair mais tráfego qualificado, ou seja, usuários interessados no conteúdo oferecido.



Arquiteturas e Padrões Avançados

- **Vantagens:**

- Experiência fluida para o usuário (sem tela piscando ou reload completo).
- Reduz carga no servidor (menos requisições completas de HTML).
- Pode funcionar offline se combinada com Service Workers.

- **Desvantagens:**

- SEO mais complexo (precisa de prerenderização ou SSR).
- Primeiro carregamento (first load) pode ser mais lento.

- **Exemplo:** Gmail, Trello, Google Docs.

- import React from 'react';
- import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
- function Home() {
- return <h1>Bem-vindo à Home</h1>;
- }
- function Sobre() {
- return <h1>Sobre Nós</h1>;
- }
- export default function App() {
- return (
- <BrowserRouter>
- <nav>
- <Link to="/">Home</Link> | <Link to="/sobre">Sobre</Link>
- </nav>
- <Routes>
- <Route path="/" element={<Home />} />
- <Route path="/sobre" element={<Sobre />} />
- </Routes>
- </BrowserRouter>
-);
- }



Arquiteturas e Padrões Avançados

- **Exemplo:**
- Para instalar os tipos do node:
 - `npm install --save-dev @types/node`
- Quando for testar:
 - <http://localhost:5173>



Arquiteturas e Padrões Avançados

- /src
 - └─ pages/
 - └─ Home.tsx
 - └─ Sobre.tsx
 - └─ main.tsx
- index.html
- tsconfig.json

```
//src/pages/Home.tsx  
import React from "react";
```

```
export default function Home(): JSX.Element {  
  return <h1>Bem-vindo à Home</h1>;  
}
```

```
//src/pages/Sobre.tsx  
import React from "react";
```

```
export default function Sobre(): JSX.Element {  
  return <h1>Sobre Nós</h1>;  
}
```



Arquiteturas e Padrões Avançados

- /src
 - |— pages/
 - | |— Home.tsx
 - | |— Sobre.tsx
 - |— main.tsx
- index.html
- tsconfig.json

```
// src/main.tsx
import React from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import Sobre from "./pages/Sobre";

const container = document.getElementById("root");

if (container) {
  const root = createRoot(container);
  root.render(
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/sobre">Sobre</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/sobre" element={<Sobre />} />
      </Routes>
    </BrowserRouter>
  );
}
```



Arquiteturas e Padrões Avançados

- /src
 - |— pages/
 - | |— Home.tsx
 - | |— Sobre.tsx
 - |— main.tsx
- index.html
- tsconfig.json

```
// index.html
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <title>SPA React TSX</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```



Arquiteturas e Padrões Avançados

- /src
 - | — pages/
 - | | — Home.tsx
 - | | — Sobre.tsx
 - | — main.tsx
- index.html
- tsconfig.json

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "jsx": "react-jsx",
    "types": ["node"],
    "moduleResolution": "Node",
    "strict": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src"]
}
```



Arquiteturas e Padrões Avançados

- **Funcionamento SPA**

- `root.render(`
- `<BrowserRouter>`
- `<nav>`
- `<Link to="/">Home</Link> | <Link to="/sobre">Sobre</Link>`
- `</nav>`
- `<Routes>`
- `<Route path="/" element={<Home />} />`
- `<Route path="/sobre" element={<Sobre />} />`
- `</Routes>`
- `</BrowserRouter>`
- `);`

- Roteamento no cliente: React Router intercepta os cliques nos `<Link>` e troca os componentes sem recarregar a página.
- Uma única página HTML (`index.html`).
- Funciona porque tudo é renderizado dentro do mesmo root e o React Router controla qual componente aparece baseado na URL.



Arquiteturas e Padrões Avançados

- **MPA (Multi Page Applications)**
- Cada rota ou página é carregada diretamente do servidor, enviando HTML completo para o cliente.
- SEO mais simples, mas a experiência pode ser menos fluída.



Arquiteturas e Padrões Avançados

- **Vantagens**

- Melhor SEO sem configurações extras (HTML já vem pronto).
- Mais simples de implementar.
- Cada página pode ser totalmente independente.

- **Desvantagens**

- Navegação menos fluida (recarregamento completo da página).
- Possível repetição de recursos (CSS/JS recarregados a cada página).

- **Exemplo:** Sites institucionais, e-commerces tradicionais.



Arquiteturas e Padrões Avançados

- **Exemplo:**
- Para instalar os tipos do node:
 - `npm install --save-dev @types/node`
- Quando for testar:
 - <http://localhost:5173>
- Obs.: o Vite não reconhece os arquivos na pasta public



Arquiteturas e Padrões Avançados

- /src
 - pages/
 - Home.tsx
 - Sobre.tsx
 - main.tsx
- sobre.html
- index.html
- tsconfig.json
- vite.config.ts

```
// src/pages/Home.tsx
import React from "react";
```

```
export default function Home(): JSX.Element {
  return <h1>Bem-vindo à Home</h1>;
}
```

```
// src/pages/Sobre.tsx
import React from "react";
```

```
export default function Sobre(): JSX.Element {
  return <h1>Sobre Nós</h1>;
}
```



Arquiteturas e Padrões Avançados

- /src
- pages/
- Home.tsx
- Sobre.tsx
- main.tsx
- sobre.html
- index.html
- tsconfig.json
- vite.config.ts

```
// src/main.tsx
import React from "react";
import { createRoot } from "react-dom/client";
import Home from "../pages/Home";
import Sobre from "../pages/Sobre";

const container = document.getElementById("root");

if (container) {
  const root = createRoot(container);

  const page = document.body.dataset.page;

  if (page === "home") {
    root.render(<Home />);
  } else if (page === "sobre") {
    root.render(<Sobre />);
  }
}
```



Arquiteturas e Padrões Avançados

- /src
 - pages/
 - Home.tsx
 - Sobre.tsx
 - main.tsx
- sobre.html
- index.html
- tsconfig.json
- vite.config.ts

// index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <title>Home</title>
  </head>
  <body data-page="home">
    <nav>
      <a href="index.html">Home</a> |
      <a href="sobre.html">Sobre</a>
    </nav>
    <div id="root"></div>
    <script type="module"
src="/src/main.tsx"></script>
  </body>
</html>
```



Arquiteturas e Padrões Avançados

- /src
- pages/
- Home.tsx
- Sobre.tsx
- main.tsx
- sobre.html
- index.html
- tsconfig.json
- vite.config.ts

// sobre.html

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <title>Sobre</title>
  </head>
  <body data-page="sobre">
    <nav>
      <a href="index.html">Home</a> |
      <a href="sobre.html">Sobre</a>
    </nav>
    <div id="root"></div>
    <script type="module"
src="/src/main.tsx"></script>
  </body>
</html>
```



Arquiteturas e Padrões Avançados

- /src
 - pages/
 - Home.tsx
 - Sobre.tsx
 - main.tsx
- sobre.html
- index.html
- tsconfig.json
- vite.config.ts

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "useDefineForClassFields": true,
    "lib": ["DOM", "DOM.Iterable", "ESNext"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "module": "ESNext",
    "moduleResolution": "Node",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx"
  },
  "include": ["src"]
}
```



Arquiteturas e Padrões Avançados

- /src
- pages/
- Home.tsx
- Sobre.tsx
- main.tsx
- sobre.html
- index.html
- tsconfig.json
- vite.config.ts

```
// vite.config.ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import { resolve } from "path";

export default defineConfig({
  plugins: [react()],
  build: {
    rollupOptions: {
      input: {
        home: resolve(__dirname,
"public/index.html"),
        sobre: resolve(__dirname,
"public/sobre.html"),
      },
    },
  },
});
```



Arquiteturas e Padrões Avançados

- **Funcionamento MPA**

- `const page = document.body.dataset.page;`

- `if (page === "home") {`
- `root.render(<Home />);`
- `} else if (page === "sobre") {`
- `root.render(<Sobre />);`
- `}`

- Roteamento no servidor: Cada HTML (index.html, sobre.html) é carregado do servidor.
- Não precisa de React Router porque cada página já é independente.
- O React só monta o componente correspondente à página atual.
- Navegação entre páginas recarrega a página inteira.

Aspecto	SPA (Single Page Application)	MPA (Multi Page Application)
Definição	Aplicação carregada em uma única página HTML , atualizada dinamicamente via JavaScript (normalmente com React, Vue, Angular).	Cada página é um arquivo HTML separado, renderizado a cada navegação (tradicional).
Carregamento inicial	Mais lento (precisa baixar todo o bundle JS).	Mais rápido (cada página é independente).
Navegação	Rápida, sem reload (usa roteamento no cliente).	Reload completo a cada troca de página.
SEO	Mais difícil (precisa de SSR ou SSG para indexação).	Mais simples (conteúdo já vem no HTML).
Performance geral	Boa depois do carregamento inicial, pois só atualiza partes da tela.	Pode ser mais pesada em apps grandes, pois recarrega todos os recursos em cada página.
Complexidade	Código mais concentrado, geralmente com frameworks modernos.	Arquitetura simples, mas pode ficar repetitiva em apps muito grandes.
Experiência do usuário	Mais fluida, parecida com um app mobile.	Menos fluida, transições de página perceptíveis.
Exemplos	Gmail, Google Drive, Trello, Facebook.	Amazon, Wikipédia, portais de notícia (NY Times, G1).
Indicado para	Dashboards, redes sociais, aplicações interativas.	E-commerces grandes, blogs, sites institucionais, portais de conteúdo.



Arquiteturas e Padrões Avançados

- **SSR (Server-Side Rendering)**
- A renderização do conteúdo acontece no servidor antes de enviar o HTML para o cliente.
- O navegador recebe a página já renderizada, mas o JavaScript ainda pode ser carregado para tornar o conteúdo interativo.
- Bom para SEO e primeira renderização rápida.



Arquiteturas e Padrões Avançados

- **Vantagens:**

- Melhor SEO e tempo de renderização inicial (TTFB e FCP).
- Conteúdo visível mais rápido para o usuário.

- **Desvantagens:**

- Maior carga no servidor.
- Configuração mais complexa.

- Métricas de desempenho
 - TFB (Time To First Byte)
 - FCP (First Contentful Paint)

- **Exemplo:** AirbNB, Reddit (versão web), Netflix (catálogo e previews)



Arquiteturas e Padrões Avançados

- **Exemplo (Next.js):**

- `export async function getServerSideProps() {`
- `return { props: { hora: new Date().toISOString() } };`
- `}`

- `export default function Home({ hora }) {`
- `return <h1>Hora atual renderizada no servidor: {hora}`
- `}`

- Essa função `getServerSideProps` é exclusiva do Next.js.
- Ela roda somente no servidor, a cada requisição.
- Tudo que você retornar em props vai ser injetado no componente da página antes de enviar o HTML para o cliente.
- O HTML já vai sair pronto com essa hora renderizada.
- Ou seja, não precisa esperar JavaScript rodar no cliente para ter conteúdo.



Arquiteturas e Padrões Avançados

- **SSG (Static Site Generation)**
- As páginas são geradas em tempo de build, antes de serem servidas.
- O servidor só entrega arquivos estáticos (HTML, CSS, JS).
- Rápido e escalável, mas não muda dinamicamente sem rebuild.



Arquiteturas e Padrões Avançados

- **Vantagens:**

- Desempenho muito alto (arquivos já prontos).
- Menos carga no servidor.
- Ótimo para SEO.

- **Desvantagens:**

- Conteúdo dinâmico exige rebuilds ou APIs externas.
- Não é ideal para dados que mudam com frequência em tempo real.

- **Exemplo:** Next.js (blogs e Docs), Páginas do GitHub, MDN Web Docs.



Arquiteturas e Padrões Avançados

- **Exemplo (Next.js):**

- `export async function getStaticProps() {`
- `return { props: { mensagem: 'Página gerada no build' } };`
- `}`

- gera o HTML uma vez no build

- `export default function Home({ mensagem }) {`
- `return <h1>{mensagem}</h1>;`
- `}`



Arquiteturas e Padrões Avançados

Recurso	<code>getServerSideProps</code> (SSR)	<code>getStaticProps</code> (SSG)
Onde roda	No servidor, a cada requisição	No build (uma vez)
Quando renderiza	Toda vez que alguém acessa	Só na compilação (ou em revalidate)
Velocidade	Mais lento (depende do servidor)	Super rápido (HTML pronto)
Exemplos de uso	Página de perfil com dados dinâmicos	Blog, landing page, documentação



Arquiteturas e Padrões Avançados

- **PWA (Progressive Web Apps)**
- Aplicações web que funcionam como apps nativos:
 - podem ser instaladas como app no celular, funcionam offline, enviam notificações e têm acesso a recursos do dispositivo.
- Usa Service Workers para cache e notificações push.
 - Service Worker é um script em JavaScript que roda em segundo plano, separado da página web.
 - Ele atua como um proxy entre a aplicação, a rede e o cache do navegador.
 - Ele intercepta requisições da página e decide se busca da rede, se entrega do cache ou se faz outra lógica.



Arquiteturas e Padrões Avançados

- **Características-chave:**
 - Manifest.json: descreve ícone, cor e nome do app.
 - Service Workers: possibilitam cache e funcionamento offline.
 - HTTPS: requisito para segurança e instalação.
- **Exemplo:** Twitter Lite, Starbucks PWA.



Arquiteturas e Padrões Avançados

• Exemplo

```
• // service-worker.js  
•  
• self.addEventListener('install', event => {  
•  
•   event.waitUntil(  
•  
•     caches.open('meu-cache').then(cache => {  
•  
•       return cache.addAll(['/index.html', '/style.css']);  
•  
•     })  
•  
•   });  
•  
•  
•  
•  
• self.addEventListener('fetch', event => {  
•  
•   event.respondWith(  
•  
•     caches.match(event.request).then(res => res || fetch(event.request)  
•  
•   );  
•  
• });
```

- install → é disparado quando o service worker é instalado pela primeira vez.
- caches.open('meu-cache') → cria (ou abre) um cache chamado "meu-cache".
- cache.addAll([...]) → salva os arquivos listados (index.html e style.css) dentro do cache.

- fetch → é disparado toda vez que a página faz uma requisição (HTML, CSS, imagens, API etc.).
- caches.match(event.request) → verifica se o recurso pedido já está guardado no cache.
- Se estiver no cache → devolve direto do cache (res). Se não estiver no cache → faz a requisição normal (fetch(event.request)).



Arquiteturas e Padrões Avançados

- **Componentização**
- Organizar a interface em partes independentes e reutilizáveis (componentes), cada uma com sua lógica, estilo e estrutura.
- Facilita manutenção e escalabilidade.



Arquiteturas e Padrões Avançados

- **Benefícios:**
 - Reutilização de código.
 - Separação de responsabilidades.
 - Facilita manutenção e testes.
- **Exemplos de bibliotecas/frameworks:**
 - React, Vue.js, Angular.



Arquiteturas e Padrões Avançados

- **Exemplo**

- ```
function Botao({ texto }) {
```
- ```
  return <button>{texto}</button>;
```
- ```
}
```

- ```
export default function App() {
```
- ```
 return (
```
- ```
    <div>
```
- ```
 <Botao texto="Salvar" />
```
- ```
      <Botao texto="Cancelar" />
```
- ```
 </div>
```
- ```
  );
```
- ```
}
```



# Arquiteturas e Padrões Avançados

- **Micro Frontends**
- Arquitetura que divide uma aplicação grande em vários pequenos frontends independentes, que se integram para formar a aplicação final.
- Cada time pode desenvolver e implantar sua parte separadamente.



# Arquiteturas e Padrões Avançados

- **Vantagens:**

- Cada equipe pode trabalhar em uma parte independente.
- Facilita escalabilidade e manutenção.
- Permite usar tecnologias diferentes em cada módulo.

- **Desvantagens:**

- Integração mais complexa.
- Maior cuidado com performance e consistência visual.

- **Exemplo:** Uma plataforma de e-commerce onde o carrinho, busca e área de usuário são micro frontends separados.



# Arquiteturas e Padrões Avançados

- **Exemplo:**

- `// webpack.config.js de um micro frontend`
- `plugins: [`
- `new ModuleFederationPlugin({`
- `name: 'app1',`
- `filename: 'remoteEntry.js',`
- `exposes: {`
- `'./Botao': './src/Botao', // Exporta componente Botao`
- `},`
- `}),`
- `]`

- `filename: 'remoteEntry.js'` Arquivo que será gerado após o build.
- Esse arquivo contém todas as informações de exportação e será consumido por outros apps que querem usar seus módulos.exposes
- Define o que esse micro frontend vai disponibilizar para outros.
  - Um botão neste exemplo



| Arquitetura / Padrão | Vantagens                                                  | Desvantagens                                    | Casos de Uso Ideais                               | Exemplo Rápido (Tecnologia)          |
|----------------------|------------------------------------------------------------|-------------------------------------------------|---------------------------------------------------|--------------------------------------|
| SPA                  | - Navegação rápida e fluida                                | - SEO pode ser desafiador                       | Apps interativas e dinâmicas (ex: Gmail, Trello)  | React Router, Vue Router, Angular    |
|                      | - Menos requisições completas                              | - Primeiro carregamento maior                   |                                                   |                                      |
| MPA                  | - SEO simples (HTML estático entregue do servidor)         | - Navegação “quebra” com reload completo        | Sites institucionais, e-commerce tradicional      | Servidor Express, PHP, Ruby on Rails |
|                      | - Implementação simples                                    | - Possível repetição de recursos em cada página |                                                   |                                      |
| SSR                  | - Conteúdo renderizado no servidor (SEO e TTFB melhorados) | - Carga maior no servidor                       | Sites e apps que precisam de SEO + interatividade | Next.js, Nuxt.js, Remix              |
|                      | - Primeira renderização rápida                             | - Complexidade maior                            |                                                   |                                      |

|                 |                                        |                                                    |                                                    |                                        |
|-----------------|----------------------------------------|----------------------------------------------------|----------------------------------------------------|----------------------------------------|
| SSG             | - Alta performance (HTML pré-gerado)   | - Não recomendado para conteúdo que muda rápido    | Blogs, documentação, landing pages                 | Next.js (getStaticProps), Gatsby, Hugo |
|                 | - Menor carga no servidor              | - Necessita rebuild para conteúdo atualizado       |                                                    |                                        |
| PWA             | - Funciona offline                     | - Pode ser complexo para implementar corretamente  | Apps web com experiência nativa (ex: Twitter Lite) | Service Worker, Manifest JSON          |
|                 | - Pode ser instalado no dispositivo    | - HTTPS obrigatório                                |                                                    |                                        |
| Componentização | - Reutilização e organização           | - Pode complicar se abusar de componentes pequenos | Qualquer app complexa                              | React, Vue, Angular                    |
|                 | - Facilita manutenção                  |                                                    |                                                    |                                        |
| Micro Frontends | - Equipes independentes                | - Integração complexa                              | Grandes plataformas modulares                      | Webpack Module Federation, Single SPA  |
|                 | - Escalabilidade e deploy independente | - Pode afetar performance se não gerenciado bem    |                                                    |                                        |



# Arquiteturas e Padrões Avançados

- **Lista 1**